QATIP Intermediate
Azure Lab03a
Service Principles

Contents

Overview 1
Before You Begin 1
Phase 0 – Create a Limited-Service Principal 2
Phase 1 – Run Terraform With Limited SP Identity 3
Phase 2 – Assign Contributor Role to the Service Principal 4

Overview

This lab introduces secure, least-privilege access using **Azure Service Principals (SPs)**. You will:

- Create a Service Principal with limited permissions scoped to the subscription
- Use SP credentials via environment variables
- Run Terraform operations and observe RBAC restrictions
- Upgrade permissions and reattempt deployments

Before You Begin

1. Ensure you have Owner or User Access Administrator permissions at the **subscription** level.

2. Azure CLI and Terraform must be installed.
3. Navigate to the lab directory:

   cd c:\azure-tf-int\labs\03a

Phase 0 – Create a Limited-Service Principal

Purpose

What we are doing:
We are creating a new **non-human identity** (Service Principal) that will be used to run Terraform, representing a **limited automation identity** — like what you would use in CI/CD pipelines.

Why it matters:
Rather than using our full-access lab account, we want to simulate **realistic access boundaries** — mimicking how secure organizations prevent developers and automation tools from having unrestricted rights.

Environment variable usage:
Instead of hardcoding secrets, we inject them into the environment. This aligns with DevSecOps best practices and is supported natively by the Terraform AzureRM provider.

Steps

1. Navigate to **terraform.tfvars** and substitute <random unique suffix> with random letters and/or numbers of your choice to ensure name uniqueness.
2. Update line 2 of **main.tf**, replacing **<your subscription id>** with your lab subscription id.
3. Create the Service Principal scoped to your subscription (replace **<sub_id>** with your lab subscription and **<3 digits>** with 3 digits of your choice):

   az ad sp create-for-rbac --name terraform-sp-<3 digits> --role Reader --scopes /subscriptions/<sub_id>

4. Using the displayed output, create the following environment variables;

   $env:ARM_CLIENT_ID = "<appId>"
   $env:ARM_CLIENT_SECRET = "<password>"
   $env:ARM_SUBSCRIPTION_ID = "<your subscription id>"
   $env:ARM_TENANT_ID = "<tenant>"

5. Adopt the SP identity:

   az logout

   az login --service-principal --username $env:ARM_CLIENT_ID --password=$env:ARM_CLIENT_SECRET --tenant $env:ARM_TENANT_ID

   Then run:

   az account show

   You should see "type": "servicePrincipal" in the output.

Phase 1 – Run Terraform With Limited SP Identity

Purpose

What we are doing:
Attempting to deploy infrastructure using Terraform under this limited SP identity.

Why it matters:
This phase is expected to fail — and that is intentional. It highlights that simply authenticating to Azure is not enough; **role assignments define what you are allowed to do**. It also encourages developers to review error messages meaningfully and connect failures to permissions.

Steps

1. Run **terraform init** and then **terraform plan**
2. Expect Terraform planning to **fail** with authorization errors.

Phase 2 – Assign Contributor Role to the Service Principal

Purpose

What we are doing:
Temporarily switching back to the lab account to assign Contributor rights to the SP.

Why it matters:
This demonstrates the **RBAC model in action** — where a principal's capabilities change dynamically based on role assignments. It mirrors how platform teams might approve temporary elevated access to unblock automation or development pipelines.

Steps

1. Re-adopt your lab identity:

   az logout
   az login

2. Inserting your SP id and Subscription id, run:

   az role assignment create --assignee <appId> --role Contributor --scope /subscriptions/<sub_id>

3. Re-adopt the SP identity:

   az logout

   az login --service-principal --username $env:ARM_CLIENT_ID --password=$env:ARM_CLIENT_SECRET --tenant $env:ARM_TENANT_ID

4. Retry:

   terraform plan
   terraform apply

5. Terraform should now **succeed**.

6. Run **terraform destroy** to remove the resources.

7. Identify the App registration object id...

   az ad sp show --id <appid> --query "{ objectId:id}" -o table

8. Attempt to delete the Service Principal...

   az ad sp delete --id <object id>

9. Attempt to delete the App registration...

   az ad app delete --id <appid>

10. The deletion attempts should fail because the Service Principal being used does not have sufficient permissions.

11. Re-adopt your lab identity:

    az logout
    az login

12. Delete the Service Principal...

    az ad sp delete --id <object id>

13. Delete the App registration...

    az ad app delete --id <appid>

14. Delete local environment variables...

    Remove-Item Env:ARM_*

Congratulations, you have completed this lab

Copyright